

Audit, Incident and Rebuild

What we found when we inspected the cluster, how we lost the whole cluster state, how we proved the root cause with evidence, and how we rebuilt it from a file in Git

WHY THIS IS GREAT INTERVIEW MATERIAL

Interviewers ask "tell me about an incident you handled". This is a real one: symptoms, evidence, a wrong first hypothesis, root cause, recovery, and what changed afterwards. Learn to tell it in 2 minutes (section 7).

Contents

1. Why audit before building
2. What the audit found
3. Fixing the disk (LVM) and cleaning up
4. The incident: 403 Forbidden for the admin
5. The investigation: evidence, not guesses
6. Root cause and contributing factors
7. How to tell this story in an interview
8. The rebuild: a cluster defined in Git
9. Lessons and habits
10. Quiz and answer key

Events of 2026-09-23 · written 2026-09-25

1. Why audit before building

An architect never deploys onto infrastructure they haven't inspected. Before writing one YAML we need the facts: **which version? what's running? how much capacity is really there? which restrictions (taints) exist?**

THE FIRST LESSON CAME IMMEDIATELY

The plan assumed 2 GB workers. The audit measured **4 GB**. And it found a problem nobody had mentioned: **disk**. Always trust measured data over what anyone remembers, including yourself.

2. What the audit found

Item	Found	Verdict
Kubernetes	v1.35.8, client = server	✓ no version skew
Container runtime	CRI-O (not containerd)	i use <code>crictl</code> on nodes
CNI	Calico manifest install, IPIP Always	✓ NetworkPolicy supported
Pod CIDR	172.17.0.0/16	⚠ same as Docker's bridge: never install Docker on nodes
RAM	3.8 GiB per node, ~3.58 GiB allocatable	✓ much better than the 2 GB assumed
Disk (root)	9.8–12 GB, 69 % / 78 % / 83 % used	✗ blocker
Taints	cka-w2: <code>node.kubernetes.io/disk-pressure:NoSchedule</code>	✗ cka-w2 can't take new pods; everything crowds onto cka-w1
StorageClass / LoadBalancer	none	i normal on kubeadm (Phases 5 and 7)
Leftovers	Broken Argo CD, 8 practice nginx pods	⚠ wasting RAM and disk
Networking	2 NICs; ens34 at /8 overlapping the Service CIDR; node IPs on DHCP	⚠ fixed (study notes 2)

3. Fixing the disk and cleaning up

3.1 Why the disks were small

The Ubuntu installer with LVM gives the root volume only about **half** the disk by default and leaves the rest unused. We added a **40 GB disk (sdb)** to each VM and joined it to the volume group.

LVM in plain words: physical disks (PV) are poured into one big pool (VG); from the pool you cut logical volumes (LV) that the filesystem sits on. Adding a disk means making it a PV, adding it to the pool, and growing the LV.

LV: ubuntu-lv → ext4 filesystem mounted at /

`lvextend -r (LV + filesystem)`

VG: ubuntu-vg (the pool)

`vgextend ubuntu-vg /dev/sdb`

PV: sda3 (old disk)

PV: sdb (new 40 GB)

`pvcreate /dev/sdb`

```
sudo pvcreate /dev/sdb                                # 1. make the new disk an LVM physical
volume
sudo vgextend ubuntu-vg /dev/sdb                      # 2. add it to the pool
sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv # 3. grow the volume; -r also grows the
filesystem, online
df -h /
```

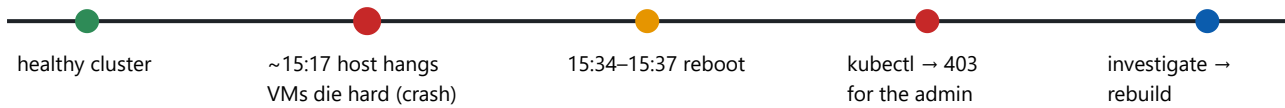
Result: 58–67 GB per node at ~13 % used. cka-w2's disk-pressure taint **cleared by itself**, because Kubernetes owns condition taints.

3.2 Cleanup, and a scope lesson

Deleting the `argocd` namespace left **3 CRDs and 6 ClusterRoles/Bindings** behind. They're **cluster-scoped**, so they don't live in any namespace. We deleted them separately.

4. The incident: 403 Forbidden for the admin

unclean shutdown on an HDD, no etcd backup



- The Windows laptop ran out of resources and **hung**. All three VMs died without a clean shutdown (`last -x` showed `crash`).
- After the restart, `kubectl` returned **403 Forbidden** for `kubernetes-admin`, both from `cka-m` and from Windows.

5. The investigation: evidence, not guesses

Evidence	What it meant
<code>kubectl auth whoami</code> worked; group <code>kubeadm:cluster-admins</code> ; binding <code>kubeadm:cluster-admins</code> NotFound	403 = authentication OK, RBAC denied . Since <code>kubeadm 1.29</code> , <code>admin.conf</code> gets its power from that binding (not <code>system:masters</code>)
Every ClusterRole/Binding, and the namespaces <code>default</code> / <code>kube-system</code> , had a creation time of 15:36 today	The API server had re-created its built-in objects on an almost empty etcd
Only 4 static pods left; no <code>calico-node</code> , <code>CoreDNS</code> , <code>kube-proxy</code> ; no CRDs; <code>cka-m</code> lost its control-plane label	All state that wasn't built-in was gone
<code>kube-system</code> can't be deleted through the API	Rules out a human <code>kubectl delete</code> : data vanished underneath the API server
etcd log: DB in use only 25 kB; <code>consistentIndex 80323 > WAL commit-index 80308</code>	The database was ahead of its own write-ahead log . Impossible if saved writes are really durable → acknowledged writes were lost
etcd log: opening the DB took 2.8 s; VM disks on E: (HDD)	Slow storage under etcd

A MISTAKE IN THE INVESTIGATION, RECORDED HONESTLY

The first hypothesis blamed the Argo CD cleanup ("a delete must have removed the binding"). That was wrong. **One missing object is not the same as a targeted delete**. Checking the creation timestamps of *everything* showed the real picture: the whole store had been reset.

6. Root cause and contributing factors

ROOT CAUSE

The host hung, the VMs were hard-powered-off, and writes that etcd believed were saved were lost (host/VMware write caching on an HDD). etcd was left internally inconsistent. **One etcd member + no backup = total loss of cluster state.**

Contributing factors

- **Host memory:** 16 GB laptop, 3 VMs × 4 GB. Commit charge 14.7 of 18.1 GB with only the VMs running, so VS Code and a browser could push it over the edge.
- **No etcd backup** existed.
- VMs weren't shut down before heavy use of the host.

What we changed

Change	Why
Windows page file 8–16 GB on the SSD (commit limit now ~24 GB)	More headroom before the host hangs
VMware "fit all VM memory into reserved host RAM", 12 GB reserved	VMs don't get swapped out by the host
cka-m reduced to 3 GB	Less memory pressure on the host
Clean shutdown order w2 → w1 → m before heavy host use	No more hard power-offs
Automated etcd backups + restore drill (study notes 4)	Next time, restore instead of rebuild

Interesting finding afterwards: the WAL fsync average was **1.5 ms** on an HDD, which is "too good". Most likely VMware acknowledges fsync from its write cache before the data reaches the disk. That's fast, and it's exactly why acknowledged writes were lost in the crash. We kept the disks and got protection from off-node backups and clean shutdowns instead.

7. How to tell this story in an interview (STAR)

Part	What to say
Situation	"In my kubeadm lab, the host hung and all VMs were powered off hard. Afterwards the admin got 403 on every request."
Task	"Find out why, restore a working cluster, and make sure it can't happen the same way again."
Action	"whoami showed authentication was fine, so it was RBAC. The kubeadm:cluster-admins binding was missing. My first guess, an accidental delete, was wrong: creation timestamps showed every built-in object had just been re-created, and the etcd log showed the DB index ahead of the WAL, meaning acknowledged writes were lost. With one etcd member and no backup, repair wasn't trustworthy, so I rebuilt from a versioned kubeadm config."
Result	"The cluster came back identical, from Git. I added automated etcd snapshots with a systemd timer, copies off the node, and a tested restore drill, plus host memory fixes and a shutdown procedure."

8. The rebuild: a cluster defined in Git

We rebuilt instead of repairing, because an etcd store in an unknown state produces bugs you can never explain. Nothing of GaramChai was deployed yet, so nothing was lost that mattered.

8.1 What `kubeadm reset` keeps and wipes

Survives	Wiped
OS, CRI-O, kubeadm/kubelet/kubectrl packages, cached images, <code>/etc/default/kubelet</code>	<code>/etc/kubernetes</code> (certs, kubeconfigs, manifests), <code>/var/lib/etcd</code> , kubelet state

Not cleaned by reset, so we did it ourselves: `/etc/cni/net.d`, `/var/lib/calico`, iptables rules, `~/.kube`. Then a reboot, for a clean slate.

8.2 The steps (R1–R10)

Step	What	Why
R1	Split-horizon <code>k8s-api</code> in both hosts files	A stable API name for nodes and Windows
R2	<code>kubeadm config validate</code> before destroying anything	It caught 2 real problems (below)
R3	Reset all nodes, workers first, and reboot	Clean slate
R4	<code>kubeadm init --config kubeadm-config.yaml</code>	Repeatable build
R5	Check the API server certificate SANs (<code>k8s-api</code> , <code>10.0.0.100</code> , <code>10.96.0.1</code>)	TLS works from everywhere
R6	Calico v3.32.0 with pool <code>172.17.0.0/16</code> and <code>IP_AUTODETECTION_METHOD=cidr=10.0.0.0/24</code>	Same network as before, pinned NIC
R7	Join the workers	—
R8	Verify: DNS (FQDN) and pod-to-pod over TCP	Prove the whole path
R9	Windows kubectrl via <code>https://k8s-api:6443</code> , no TLS workaround	—
R10	etcd backups: manual → off the node → automated + restore drill	Never again

8.3 Validation saved us twice

1. The old config (from Sep 6) had a **duplicate key** (`serializeImagePulls`) that kubeadm 1.35 rejects in strict mode. A rebuild with it would have failed half-way through `init`.
2. The new config started with `---` after the header comments, which creates an **empty first document** ("kind and apiVersion is mandatory"). `---` goes only *between* documents.

HABIT

Validate (`kubeadm config validate`, `kubectrl apply --dry-run=server`) **before** anything destructive.

9. Lessons and habits

Lesson	Habit
Measured data beat the plan (2 GB vs 4 GB)	Audit first, always
One missing object $\neq$ a targeted delete	Check timestamps of <i>everything</i> before naming a cause
No backup = no recovery	Back up etcd and the PKI from day 1, off the node, and test restores
Hard power-offs lose "saved" data	Shut down cleanly: w2 $\rightarrow$ w1 $\rightarrow$ m
Output didn't match "expect" and we carried on	Output $\neq$ expected $\rightarrow$ stop and ask
A per-node value was pasted on the wrong node (-- node-ip)	Check per-server values before pressing Enter
~ is /root for root, /home/redhat for redhat	Use full paths with sudo; don't work in a root shell
Static-pod restart counts were inherited from before the reset	Check when restarts happened
CoreDNS Pending: it tolerated not-ready only as NoExecute	A toleration must match key and effect

10. Quiz

1. Name two findings from the audit that contradicted what we believed before.
2. Explain LVM's PV, VG and LV using the pool analogy, and the three commands used to add a disk.
3. Why did cka-w2's taint disappear without anyone removing it?
4. Deleting the argocd namespace left objects behind. Which kind, and why?
5. The admin gets 403. `kubectl auth whoami` works. What does that tell you immediately?
6. Which single piece of evidence ruled out "someone deleted it with kubectl"?
7. What did `consistentIndex 80323 > commit-index 80308` tell us?
8. What was wrong with the first hypothesis, and what's the general lesson?
9. State the root cause in one sentence.
10. Why did we rebuild instead of trying to repair etcd?
11. What does `kubeadm reset` NOT clean up?
12. Name the two problems `kubeadm config validate` caught.
13. Why is `controlPlaneEndpoint` set to `k8s-api:6443`?
14. Why was a 1.5 ms fsync on an HDD suspicious rather than good news?
15. Tell the incident as a STAR story in four sentences.

Answer key

1. Workers had 4 GB, not 2 GB; disk (not RAM) was the real problem; the runtime was CRI-O, not containerd.
2. PVs (disks) are poured into a VG (the pool); LVs are cut from the pool and carry the filesystem. `pvccreate /dev/sdb, vgextend ubuntu-vg /dev/sdb, lvextend -r -l +100%FREE ...`.
3. It's a condition taint owned by Kubernetes: once free disk rose above the threshold, the kubelet cleared DiskPressure and the controller removed the taint.
4. Cluster-scoped objects (CRDs, ClusterRoles, ClusterRoleBindings): they're not inside any namespace, so a namespace delete doesn't touch them.
5. Authentication works; authorization (RBAC) fails. Look for the missing Role/Binding.
6. `kube-system` itself had been re-created, and it can't be deleted through the API, so the data vanished underneath the API server.
7. The database was ahead of its write-ahead log: writes that had been acknowledged as durable weren't actually on disk. Storage lost data.
8. It explained one missing object with a targeted delete. Lesson: check creation timestamps of everything before naming a cause.
9. A hard power-off lost writes etcd believed were durable, leaving a single-member etcd inconsistent, and there was no backup to restore.
10. An etcd store in an unknown state produces unexplainable bugs, nothing important was deployed, and a versioned config made the rebuild repeatable.
11. `/etc/cni/net.d, /var/lib/calico, iptables rules, users' ~/.kube`.
12. A duplicate `serializeImagePulls` key in the old config; a leading `---` creating an empty first document.
13. A stable name (not a node IP) that can later point at a load balancer for HA; it's baked into certs and kubeconfigs and hard to change after init.
14. An HDD can't really persist in 1.5 ms; the host was acknowledging from its write cache, which is exactly how "saved" writes were lost in the crash.
15. See section 7: host hang → admin 403; whoami showed RBAC; timestamps + etcd index proved storage loss; rebuilt from Git and added tested backups and host fixes.